RICE UNIVERSITY

T1 · Train a next-token model in NumPy

Retained prototype: loss, gradients, optimizer and training loop in NumPy. The syllabus T1 differs.

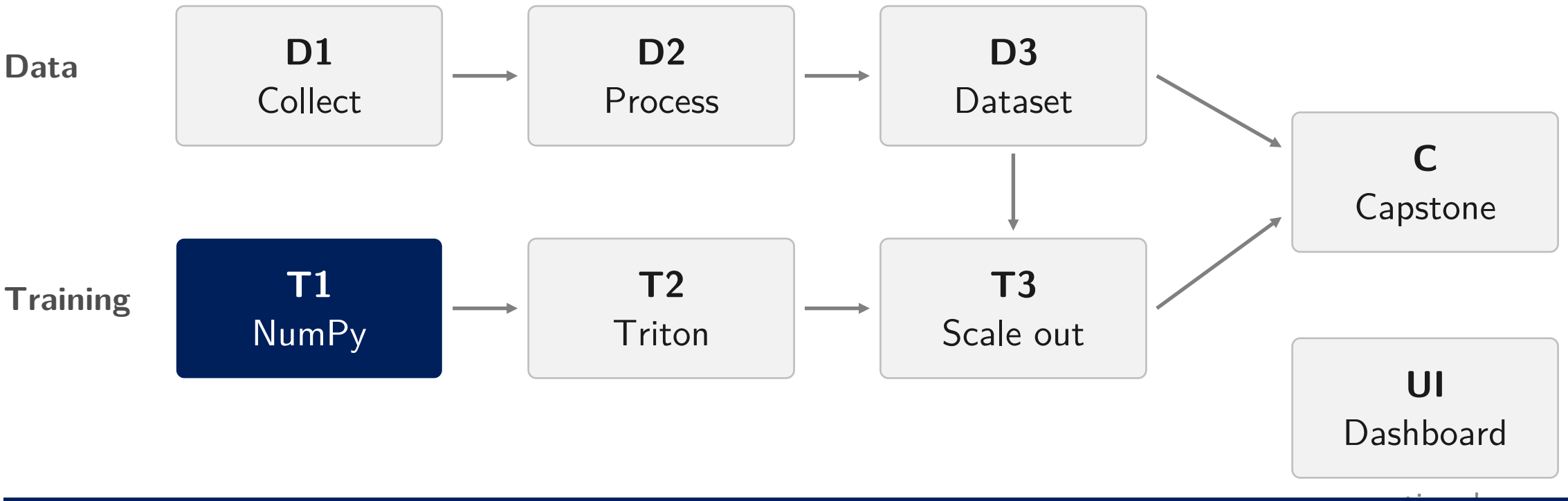
COMP 329 course staff

Rice University

Spring 2027 · Assignment T1

Release week 8 · Checkpoint week 9 · Final week 10 · 12-15 hours

Where this assignment fits



In our pipeline

This assignment: `runs/inputs/byte` → **T1** → `checkpoint.npz`

What you will build, and why

Given the previous C tokens, predict the next one, and understand every array along the way.

1 **See the math**

A tiny MLP lets you inspect every shape, value and derivative.

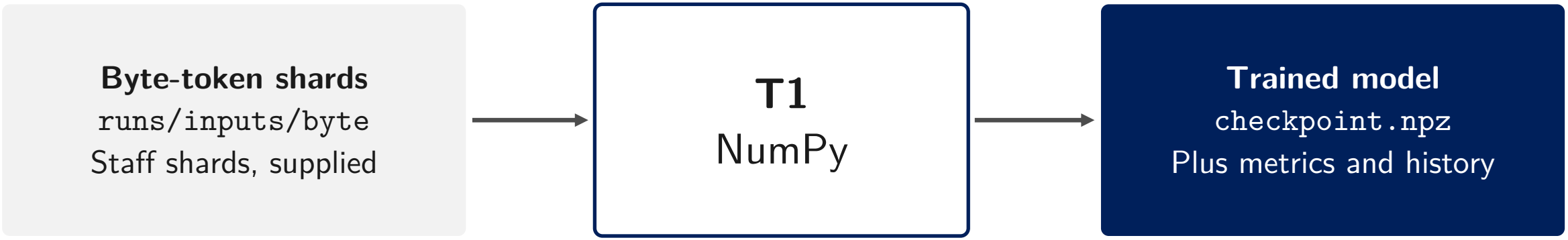
2 **Correct first**

A gradient checker and known optimizer updates catch mistakes early.

3 **Reused later**

T2 runs this model on a GPU; T3 runs this training loop on many workers.

What goes in, what comes out



Files you produce

`metrics.json`
Final validation and test scores

`history.json`
Loss and validation bits per byte over time

`checkpoint.npz`
Weights, optimizer moments, loader position

Follow the shapes through the model

1

contexts (B, C)
Integer token IDs, never one-hot.

→

2

embed (B, C, D)
Look up rows of E (V, D).

→

3

flatten (B, C · D)
One vector per example.

→

4

hidden (B, H)
tanh(x @ W1 + b1)

→

5

logits (B, V)
hidden @ W2 + b2

Key idea

Example: B=2, C=3, D=2, H=4, V=9 gives x (2, 6), hidden (2, 4), logits (2, 9). Write these beside your code before backward.

Stable loss, then backward

```
m = logits.max(axis=1, keepdims=True)
z = logits - m           # no overflow
logZ = log(exp(z).sum(1)) + m
loss = logZ - logits[i, y]

dlogits = softmax(logits)
dlogits[i, y] -= 1
dlogits /= B             # once!
```

1 **Subtract the row max**

exp(1000) overflows; the probabilities do not change.

2 **Divide by B once**

The loss is a mean over the batch.

3 **Affine backward**

dW = x.T @ dz, db = dz.sum(0), dx = dz @ W.T

4 **Repeated IDs add up**

Context [1, 1, 2]: both copies of ID 1 add into E[1]. Accumulate; don't overwrite.

Three optimizers, one update rule

SGD

`w ← w − lr · g`

Start here.

Momentum

$v \leftarrow \mu v + g$

`w ← w − lr · v`

Stores one velocity per parameter.

AdamW

`m, v moments (0.9, 0.999)`

bias correction, $\varepsilon = 1e\text{-}8$

Stores two moments per parameter.

Key idea

Decoupled weight decay for all three: multiply E and W* by $(1 - \text{lr} \cdot \text{decay})$, then subtract $\text{lr} \cdot \text{update}$. Biases are exempt. Weight 1, gradient 2, lr 0.1, decay 0.2 $\rightarrow$ 0.78.

What you write, what you get

You write

`cross_entropy`, `MLP.forward`, `MLP.loss_and_grad`

`starter/model.py`

`Optimizer.step`

`starter/optim.py`

`train`

`starter/training.py`

Provided

- Dataset and Loader
- Initialization, clipping, evaluation
- Checkpoint I/O and generation
- Gradient checker

Ground rules

Keep the signatures · don't edit `starter/pipeline_lab` · run through `course.py` or `lab.py`.

Build it in this order

1

Stable loss

Row-max subtraction and the mean gradient.

2

Forward + backward

Every layer, including repeated IDs.

`check t1 --checkpoint`

3

Optimizer

SGD, then momentum, then AdamW.

4

Training loop

Evaluate, checkpoint, resume exactly.

`check t1`

5

Measure

Memory prediction, batch and width sweeps.

`lab.py t1`

Check

Done when: All 6 tests pass (including the 2 checkpoint tests); resumed training is bit-identical.

If something goes wrong

NaN loss
Check: Row-max subtraction, the log-normalizer, the gradient norm.

Shape mismatch in backward
Check: Trace (B, V) $\rightarrow$ (B, H) $\rightarrow$ (B, C $\cdot$ D).

Only the embedding gradient fails
Check: Repeated IDs must add, not overwrite.

Resume differs
Check: Loader position, moments, seed; steps is a total.

Gradient off by a factor of B
Check: Divide by B exactly once.

GPU reuse fails later (T2)
Check: Use self.xp and self.mm, not hard-coded NumPy.

The performance study

How do batch size and width change memory and speed?

Predict first

Parameters $= V \cdot D + C \cdot D \cdot H + H + H \cdot V + V$. Gradients and the two AdamW moments each add a parameter-sized buffer.

Batch 17 / 64 / 128
fixed model

Hidden 16 / 32 / 64
fixed batch

How to measure: One warmup + three repeats. Report targets/s and training vs. setup time. Equal steps with different batches are different work; say so.

How you are graded

20	40	16	24
Checkpoint	Final correctness	Performance	Analysis

Correctness: loss/forward 8 · gradients 12 · optimizer/clipping 10 · loop/evaluation/resume 10

Check

Done when: All 6 tests pass (including the 2 checkpoint tests); resumed training is bit-identical.

Your first ten minutes

```
$ bash setup.sh
$ source .venv/bin/activate
$ python course.py preflight
$ python course.py inputs --output runs/inputs
$ python course.py check t1 --checkpoint
$ python lab.py t1 --input runs/inputs/byte \
    --output runs/t1-first --steps 40
```

Your folder

`04_t1_numpy/`

Handout:
`04_t1_numpy/README.md`